Data Structures in R & Python

Lecture 06

Dr. Colin Rundel

Tuples

Tuples

Python tuples are *heterogeneous*, *ordered*, **immutable** containers - lists whose contents cannot be changed. R has no tuple type, a list (or an atomic vector) is used instead.

```py
(1, 2, 3)
```

```
(1, 2, 3)
```

```py
(1, True, "abc")
```

```
(1, True, 'abc')
```

```py
type( (1) )
```

```
<class 'int'>
```

```py
type( (1,) )
```

```
<class 'tuple'>
```

```py
x = (1, 2, 3)
x[2] = 5
```

```
TypeError: 'tuple' object does not support
```

```py
x = (1, [2, 3])
x[1].append(4)
x
```

```
(1, [2, 3, 4])
```

It is the *comma* that creates a tuple, not the parentheses - `(1)` is just the number 1 and a one element tuple must be

Unpacking

Assigning a sequence to several names at once *unpacks* it - this is how Python functions return multiple values. A name prefixed with $*$ collects any remaining values into a list.

```py
def minmax(x):
    return min(x), max(x)
minmax([3, 1, 2])
```

```
(1, 3)
```

```py
lo, hi = minmax([3, 1, 2])
print(lo, hi)
```

```
1 3
```

```py
x, y = 1, 2
x, y = y, x
x, y
```

```
(2, 1)
```

```py
first, *rest = range(6)
first
```

```
0
```

```py
rest
```

```
[1, 2, 3, 4, 5]
```

```py
pairs = [("a", 1), ("b", 2)]
for i, (k, v) in enumerate(pairs):
    print(i, k, v)
```

```
0 a 1
1 b 2
```

The number of names must match the length of the sequence (`x, y = [1, 2, 3]` is a `ValueError`) unless one of them is

Dictionaries

Dictionaries

Python `dict`s are *mutable* containers of **key: value** pairs, designed for the efficient lookup of a value by its key. The closest R analog is a named list (or a named atomic vector when the values are homogeneous).

```py
{"abc": 123, "def": 456}
```

```
{'abc': 123, 'def': 456}
```

```py
dict([("abc", 123), ("def", 456)])
```

```
{'abc': 123, 'def': 456}
```

```py
dict(abc=123, hello=456)
```

```
{'abc': 123, 'hello': 456}
```

```py
dict(zip(["abc", "def"], [1, 2]))
```

```
{'abc': 1, 'def': 2}
```

```R
list(abc = 123, def = 456)
```

```
$abc
[1] 123

$def
[1] 456
```

```R
c(abc = 123, def = 456)
```

```
abc def
123 456
```

Since Python 3.7 dictionaries preserve insertion order. The keyword form of `dict()` only works for keys that are valid

Keys must be hashable

Keys can be any *immutable* (technically, *hashable*) object - numbers, strings, tuples of immutable objects. Values can be anything.

Immutable	Mutable
`int`, `float`, `bool`, `complex`	`list`
`str`, `bytes`	`dict`
`tuple`, `frozenset`, `range`	`set`
`None`	instances of your own classes

Using a mutable object as a key is an error, even when it is nested inside a tuple,

```py
1 {[1]: "bad"}
```

```
TypeError: cannot use 'list' as a dict key
```

```py
1 {(1, [2]): "bad"}
```

```
TypeError: cannot use 'tuple' as a dict ke
```

`hash()` maps an object to an integer that is used to locate the key in memory - a mutable object's hash could change

Lookup

`[]` raises a `KeyError` for a missing key, `in` tests for a key and `.get()` supplies a default.

```py
x = {1: "abc", "y": "hello", (1, 1): 3.14}
x[1]
```

```
'abc'
```

```py
x[(1, 1)]
```

```
3.14
```

```py
x["def"]
```

```
KeyError: 'def'
```

```py
"y" in x
```

```
True
```

```py
"hello" in x
```

```
False
```

```py
print( x.get("def") )
```

```
None
```

```py
x.get("def", 0)
```

```
0
```

Insert, replace, remove

Assigning to a key inserts or replaces, `del` and `.pop()` remove. Note that assigning `None` does *not* remove a key, whereas in R assigning `NULL` does.

```py
x = {1: "abc", "y": "hello"}
x["def"] = -1
x["y"] = "goodbye"
x
```

```
{1: 'abc', 'y': 'goodbye', 'def': -1}
```

```py
del x[1]
x.pop("def")
```

```
-1
```

```py
x["y"] = None
x
```

```
{'y': None}
```

```R
y = list(abc = 123, def = 456)
y$ghi = -1
y$def = "goodbye"
str(y)
```

```
List of 3
 $ abc: num 123
 $ def: chr "goodbye"
 $ ghi: num -1
```

```R
y$abc = NULL
str(y)
```

```
List of 2
 $ def: chr "goodbye"
 $ ghi: num -1
```

Methods & iteration

Iterating over a `dict` yields its *keys* - use `.items()` to get keys and values together.

```py
x = {1: "abc", "y": "hello"}
x.keys()
```

```
dict_keys([1, 'y'])
```

```py
x.values()
```

```
dict_values(['abc', 'hello'])
```

```py
x.items()
```

```
dict_items([(1, 'abc'), ('y', 'hello')])
```

```py
for k, v in x.items():
    print(k, "->", v)
```

```
1 -> abc
y -> hello
```

```py
x | {"y": "goodbye", "w": 0}
```

```
{1: 'abc', 'y': 'goodbye', 'w': 0}
```

```py
x.update({"y": "goodbye", "w": 0})
x
```

```
{1: 'abc', 'y': 'goodbye', 'w': 0}
```

`.keys()`, `.values()`, and `.items()` return dynamic *views* of the dictionary rather than copies - see the docs. `|` returns a

Environments

Environments

An environment is a set of name-object *bindings* - the workspace is one, each package is one, and every function call creates a fresh one for its local variables. A lookup that fails in an environment continues in its *parent*.

```r
x = 1
f = function() {
  y = 2
  environment()
}
e = f()
ls(e)
```

```
[1] "y"
```

```r
parent.env(e)
```

```
<environment: R_GlobalEnv>
```

```r
get("x", envir = e)
```

```
[1] 1
```

```r
g = function() {
  z = 3
  z
}
g()
```

```
[1] 3
```

```r
exists("z")
```

```
[1] FALSE
```

`globalenv()` is the workspace and the chain of parents ends at `emptyenv()`. Looking up through the parents is *lexical scoping* - `f()` finds `x` in the global environment, and `z` is gone once `g()` returns. More on this with functional

Environments as hash maps

Bindings are stored in a hash table, so an environment makes a good `dict` - use `parent = emptyenv()` so that lookups do not fall through to the workspace.

```r
h = new.env(parent = emptyenv())
h$a = 1
h[["b"]] = 2
```

```r
ls(h)
```

```
[1] "a" "b"
```

```r
h$a
```

```
[1] 1
```

```r
exists("x", envir = h)
```

```
[1] FALSE
```

```r
rm("a", envir = h)
ls(h)
```

```
[1] "b"
```

```r
h2 = h
h2$d = 4
ls(h)
```

```
[1] "b" "d"
```

```r
h[1]
```

```
Error in `h[1]`:
! object of type 'environment' is not subsettabl
```

Modifying `h2` changed `h` - environments have *reference* semantics, both names refer to the same object exactly like two

Exercise 1

Represent the JSON record from last lecture in Python using `dict`s and `list`s (JSON happens to be valid Python syntax), then:

```
{
  "firstName": "John",
  "lastName": "Smith",
  "age": 25,
  "address": {
    "streetAddress": "21 2nd Street",
    "city": "New York",
    "state": "NY",
    "postalCode": 10021
  },
  "phoneNumber": [
    { "type": "home",
      "number": "212 555-1239" },
    { "type": "fax",
      "number": "646 555-4567" }
  ]
}
```

- Extract the fax number.
- Add a `"mobile"` phone number and remove `age`.
- Write a function `phone(person, type)` that returns the number of the requested type, or `None` if there is no such entry.
- How would `phone()` look in R for the list version from last time?

Sets

Sets

A `set` is a *mutable*, *unordered* collection of **unique** hashable elements - there are no positions, so the primary operation is membership testing with `in`.

```py
x = {1, 2, 3, 4, 1, 2}
x
```

```
{1, 2, 3, 4}
```

```py
set("mississippi")
```

```
{'s', 'p', 'i', 'm'}
```

```py
3 in x
```

```
True
```

```py
x[0]
```

```
TypeError: 'set' object is not subscriptable
```

```py
x.add(9)
x.discard(8)
x.update([7, 8])
x
```

```
{1, 2, 3, 4, 7, 8, 9}
```

```py
x.remove(6)
```

```
KeyError: 6
```

```py
{1, 2, [1, 2]}
```

```
TypeError: cannot use 'list' as a set element (u
```

`{}` creates an empty *dictionary*, an empty set must be written `set()`. `remove()` raises a `KeyError` if the element is absent

Set operations

Sets support the usual mathematical operations. R has no set type - any vector can be treated as one via `unique()` and the set functions.

```py
x = {1, 2, 3}
y = {2, 3, 4}
x | y
```

```
{1, 2, 3, 4}
```

```r
x = c(1, 2, 3, 1)
y = c(2, 3, 4)
union(x, y)
```

```
[1] 1 2 3 4
```

```py
x & y
```

```
{2, 3}
```

```r
intersect(x, y)
```

```
[1] 2 3
```

```py
x - y
```

```
{1}
```

```r
setdiff(x, y)
```

```
[1] 1
```

```py
1 in x
```

```
True
```

```r
1 %in% x
```

```
[1] TRUE
```

Python's set methods (`x.union(y)`, `x.intersection(y)`, ...) accept any iterable, the operators require sets on both sides.

Mutability & copies

Nested containers

In Lecture 2 we saw that `.copy()` is *shallow*, and `copy.deepcopy()` copies recursively. The same sharing problem occurs when building nested structures by repetition,

```py
grid = [[0] * 3] * 3
grid[0][0] = 1
grid
```

```
[[1, 0, 0], [1, 0, 0], [1, 0, 0]]
```

```py
grid = [[0] * 3 for _ in range(3)]
grid[0][0] = 1
grid
```

```
[[1, 0, 0], [0, 0, 0], [0, 0, 0]]
```

```py
import copy

x = {"a": [1, 2], "b": [3, 4]}
y = x.copy()
z = copy.deepcopy(x)

x["a"].append(99)
y
```

```
{'a': [1, 2, 99], 'b': [3, 4]}
```

```py
z
```

```
{'a': [1, 2], 'b': [3, 4]}
```

`[[0] * 3] * 3` creates *one* inner list and references it three times. The comprehension evaluates `[0] * 3` three separate

Mutable default arguments

Default values are evaluated *once*, when the function is defined, so a mutable default is shared by every call that does not supply the argument. Use `None` as the default and create the object inside the function instead.

```py
def f(x=[]):
    x.append(1)
    return x

f()
```

```
[1]
```

```py
f()
```

```
[1, 1]
```

```py
def f(x=None):
    if x is None:
        x = []
    x.append(1)
    return x

f()
```

```
[1]
```

```py
f()
```

```
[1]
```

Algorithms & data structures

Big-O notation

Big-O notation describes the *complexity* of an algorithm - how the time (or memory) required grows with the size of the input n. Only the fastest growing term matters (constant factors and lower order terms are dropped), so two algorithms with the same Big-O can still differ greatly in practice.

Since performance depends on the data we usually quote *average* or *worst case* complexity, and *amortized* complexity when an occasional expensive step is paid for by many cheap ones (e.g. growing a list).

Complexity	Big-O
Constant	$O(1)$
Logarithmic	$O(\log n)$
Linear	$O(n)$
Quasilinear	$O(n \log n)$
Quadratic	$O(n^2)$
Exponential	$O(C^n)$

Under the hood

The containers in both languages are built from a handful of basic structures - knowing which is which explains what each container is good, and bad, at.

Structure	Layout	R	Python
Array	contiguous block of same-sized values	atomic vector	`ndarray`
Array of pointers	contiguous block of *references* to objects	list	`list`, `tuple`
Linked list	nodes that each point to their neighbors	pairlist	`deque`
Hash table	array indexed by `hash(key)`	environment	`dict`, `set`

Vectors

An atomic vector or `ndarray` is a single contiguous block of memory holding values of one type - element i sits at a fixed offset from the start, so accessing any element is O(1). The cost is that every element must have the same size.

```r
object.size(numeric(1e6))
```

```
8000048 bytes
```

```r
object.size(integer(1e6))
```

```
4000048 bytes
```

```python
np.zeros(1_000_000).nbytes
```

```
8000000
```

```python
np.zeros(1_000_000, dtype="int64").nbytes
```

```
8000000
```

```python
np.zeros(1_000_000, dtype="int32").nbytes
```

```
4000000
```

Growing a vector

Growing a vector allocates a new block and copies everything, so `c(x, i)` in a loop is $O(n^2)$ overall.

```r
n = 5e4
x = c()
system.time(
  for (i in seq_len(n)) x = c(x, i)
)
```

```
   user  system elapsed
  1.073   0.019   1.092
```

Assigning past the end grows in place and preallocating avoids the question entirely.

```r
x = numeric()
system.time(
  for (i in seq_len(n)) x[i] = i
)
```

```
   user  system elapsed
  0.004   0.000   0.004
```

```r
x = numeric(n)
system.time(
  for (i in seq_len(n)) x[i] = i
)
```

```
   user  system elapsed
  0.001   0.000   0.001
```

Generic vectors

A `list` in either language is an array of *pointers* to separately allocated objects - elements can be any type and size because the array only stores addresses, and indexing is still O(1), but every value carries its own header.

Every R object is a `SEXP`, a pointer to a struct holding

- a header with the type (`SEXPTYPE`), flags, and garbage collector bookkeeping
- a pointer to the object's attributes (a pairlist)
- the data
  - for an atomic vector (`REALSXP`, `INTSXP`, …) the length followed by a block of values
  - for a list (`VECSXP`) the length followed by an array of `SEXP`s

Every Python value is a `PyObject`, a struct holding

- a reference count (`ob_refcnt`) and a pointer to its type (`ob_type`)
- type specific data
  - a `float` stores its value
  - a `list` stores its length, its allocated capacity, and a pointer to an array of `PyObject *`

The pointer array is over-allocated so that appending is amortized O(1) - a copy is only needed when the spare slots run

Array

https://visualgo.net/en/array - R atomic vectors and NumPy arrays are contiguous blocks of same-typed values. A Python `list` is a contiguous array of *pointers* to objects, which is why it can be heterogeneous and why it uses far more

deques

A `deque` (double ended queue) from the `collections` module is a linked list, so adding or removing at *either* end is O(1) - a `list` is only efficient at the end, and reaching element i of a deque means walking the chain.

```py
from collections import deque
x = deque(range(3))
x.appendleft(-1)
x
```

```
deque([-1, 0, 1, 2])
```

```py
x.popleft()
```

```
-1
```

```py
x
```

```
deque([0, 1, 2])
```

```py
x = deque(range(3), maxlen=4)
x.append(10)
x.append(11)
x
```

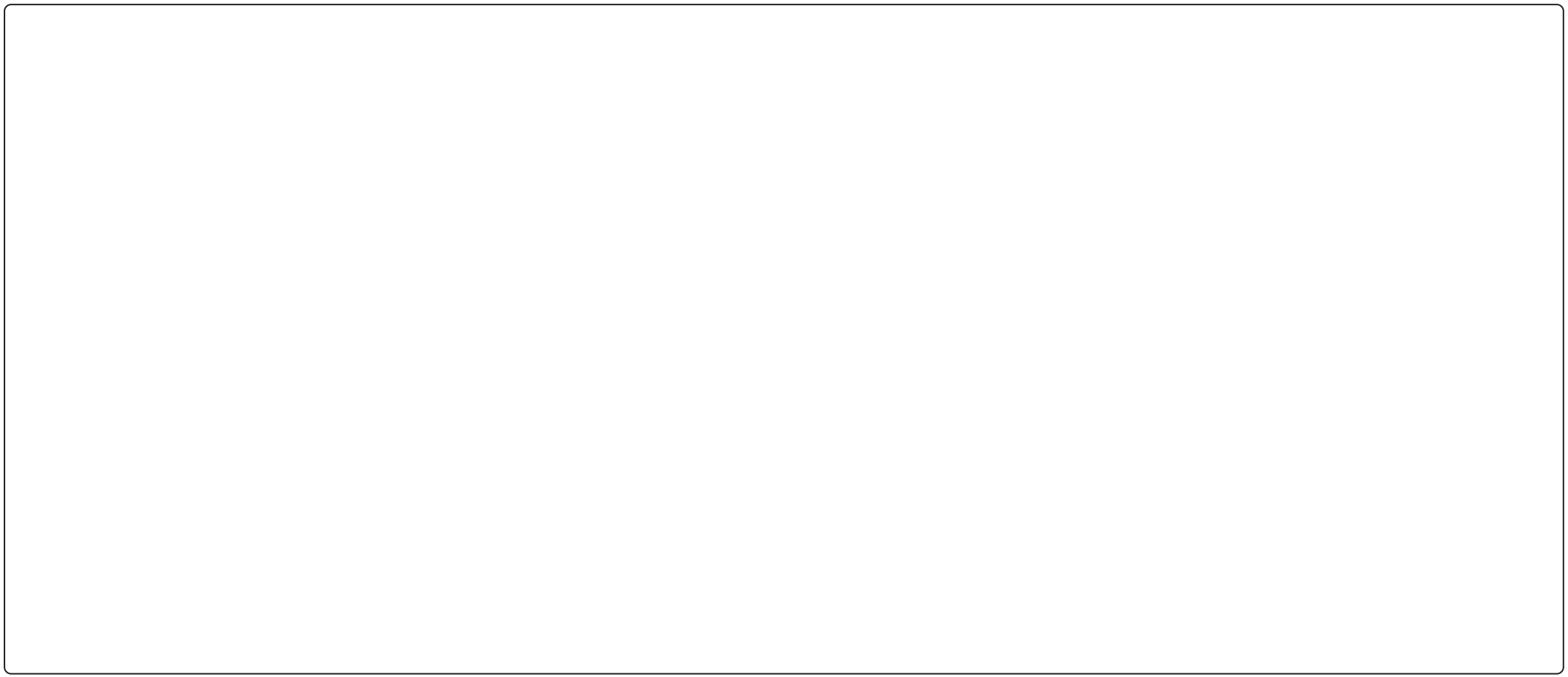
```
deque([1, 2, 10, 11], maxlen=4)
```

```py
x.appendleft(-1)
x
```

```
deque([-1, 1, 2, 10], maxlen=4)
```

With `maxlen` set, adding to a full deque drops an element from the opposite end - a rolling window of the last n items.

Linked list



https://visualgo.net/en/list - Python's deque is a doubly linked list of small arrays. R uses *pairlists* (linked lists) internally

Hash tables in practice

A hash table stores each entry in a slot chosen by `hash(key)`, so lookup is O(1) on average however many entries there are, while a `list` (or a loop over a vector) has to check each element. Measuring confirms it,

```py
import time
big_list = list(range(100_000))
big_set = set(big_list)
queries = range(0, 100_000, 100)

start = time.perf_counter()
res = [q in big_list for q in queries]
print(time.perf_counter() - start)
```

```
0.20874766599445138
```

```py
start = time.perf_counter()
res = [q in big_set for q in queries]
print(time.perf_counter() - start)
```

```
0.0014721659972565249
```

```R
big = sample(1e5)
queries = sample(1e5, 1e3)

system.time({
  for (q in queries) any(big == q)
})
```

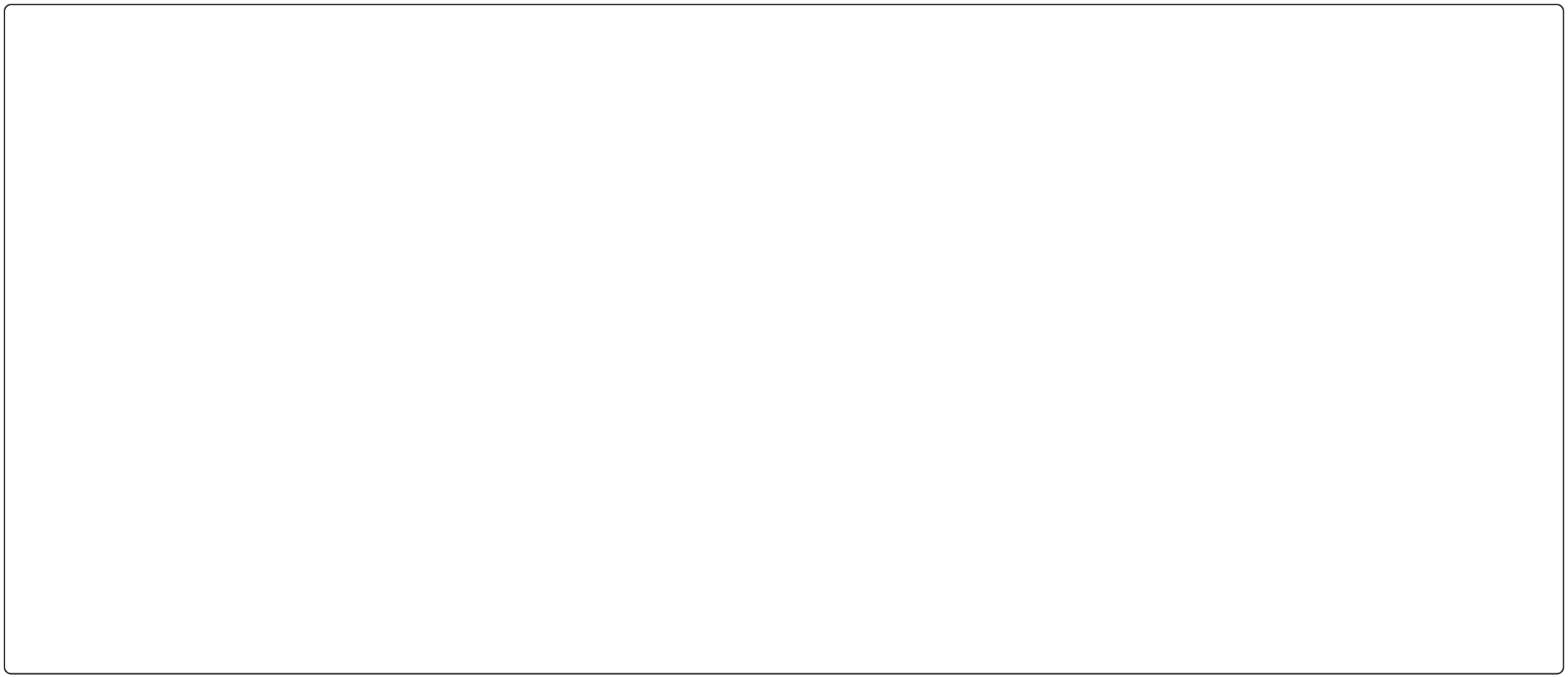
```
   user  system elapsed
  0.112   0.000   0.112
```

```R
system.time({
  queries %in% big
})
```

```
   user  system elapsed
  0.005   0.000   0.004
```

The slot array is kept sparse so that collisions stay rare, hence the extra memory. For careful timing use `timeit` (Python)

Hash table



https://visualgo.net/en/hashtable - Python's `dict` and `set` are hash tables, as are R environments. R's `match()`, `%in%`, `unique()`, and `duplicated()` build a hash table of one argument, which is why they are fast. Equal keys must hash

Time complexity in Python

Operation	list	dict / set	deque
Copy	O(n)	O(n)	O(n)
Append	O(1)	O(1)	O(1)
Insert	O(n)	O(1)	O(n)
Get item	O(1)	O(1)	O(n)
Set item	O(1)	O(1)	O(n)
Delete item	O(n)	O(1)	O(n)
`x in s`	O(n)	O(1)	O(n)
`pop()`	O(1)	O(1)	O(1)
`pop(0)`	O(n)	—	O(1)

Average case complexities, from https://wiki.python.org/moin/TimeComplexity. Appends are *amortized* O(1) - the list

Exercise 2

For each of the following scenarios, which data structure (in Python and in R) is the most appropriate and why?

- A fixed collection of 100 integers.
- A queue (first in, first out) of customer records.
- A stack (first in, last out) of customer records.
- A count of word occurrences within a document.
- The heights of the bars in a histogram with even bin widths.
- Checking whether each of a million words is in a list of 50 stop words.
- The last 10 lines read from a very large log file.

Comparing R & Python

Container summary

Python	Properties	R
`list`	ordered, mutable, heterogeneous	`list()`
`tuple`	ordered, immutable, heterogeneous	`list()` or atomic vector (copy on modify)
`dict`	key lookup, mutable, insertion ordered	named `list()`, `new.env()` for hashing
`set`	unique, unordered, mutable	vector + `unique()`, `union()`, `%in%`, …
`deque`	fast at both ends	none (use a list or vector)
`range`	lazy integer sequence, immutable	`seq_len()`, `:` (materialized)
NumPy `ndarray`	homogeneous, vectorized	atomic vector, matrix, array

Takeaways

- Tuples are immutable lists - use them for fixed groups of values and unpack them into names.
- Dictionaries map hashable keys to values with O(1) lookup, R's named lists do the same job for small tables and environments for large ones.
- Sets give O(1) membership and the standard set algebra, R provides the same via `unique()`, `union()`, `intersect()`, `setdiff()`, and `%in%`.
- Only immutable objects can be keys, shallow copies share nested objects, and mutable default arguments are shared between calls - reach for `deepcopy()` (or `None`) when in doubt.
- Know the complexity of the operations you use (a `deque` for queues, a `set` for membership, preallocation in R), and measure when it matters.